

ЛЕКЦИЯ 6

Иерархические автоматы и деревья поведения

Как описать логику миссии из сорока состояний и шести исключений и не утонуть

две части по 45

ение робототехническими комплексами»

План занятия

- 1 Где ломается плоский автомат: считаем переходы
- 2 Statecharts Харела: иерархия, ортогональность, история
- 3 Количественный выигрыш и его границы
- 4 Деревья поведения: узлы, тик, три статуса
- 5 Формальная семантика и области успеха

- 6 Теорема о безопасности Fallback
- 7 Теорема о последовательной композиции
- 8 Реактивность и её цена
- 9 Навык как поддереву: контракт становится структурой
- 10 Типовые ошибки проектирования деревьев

Синие — первая часть · янтарные — вторая часть

ЧАСТЬ 1

Плоский автомат и его предел

Считаем переходы, находим предел и разбираем первое решение — иерархию Харела.

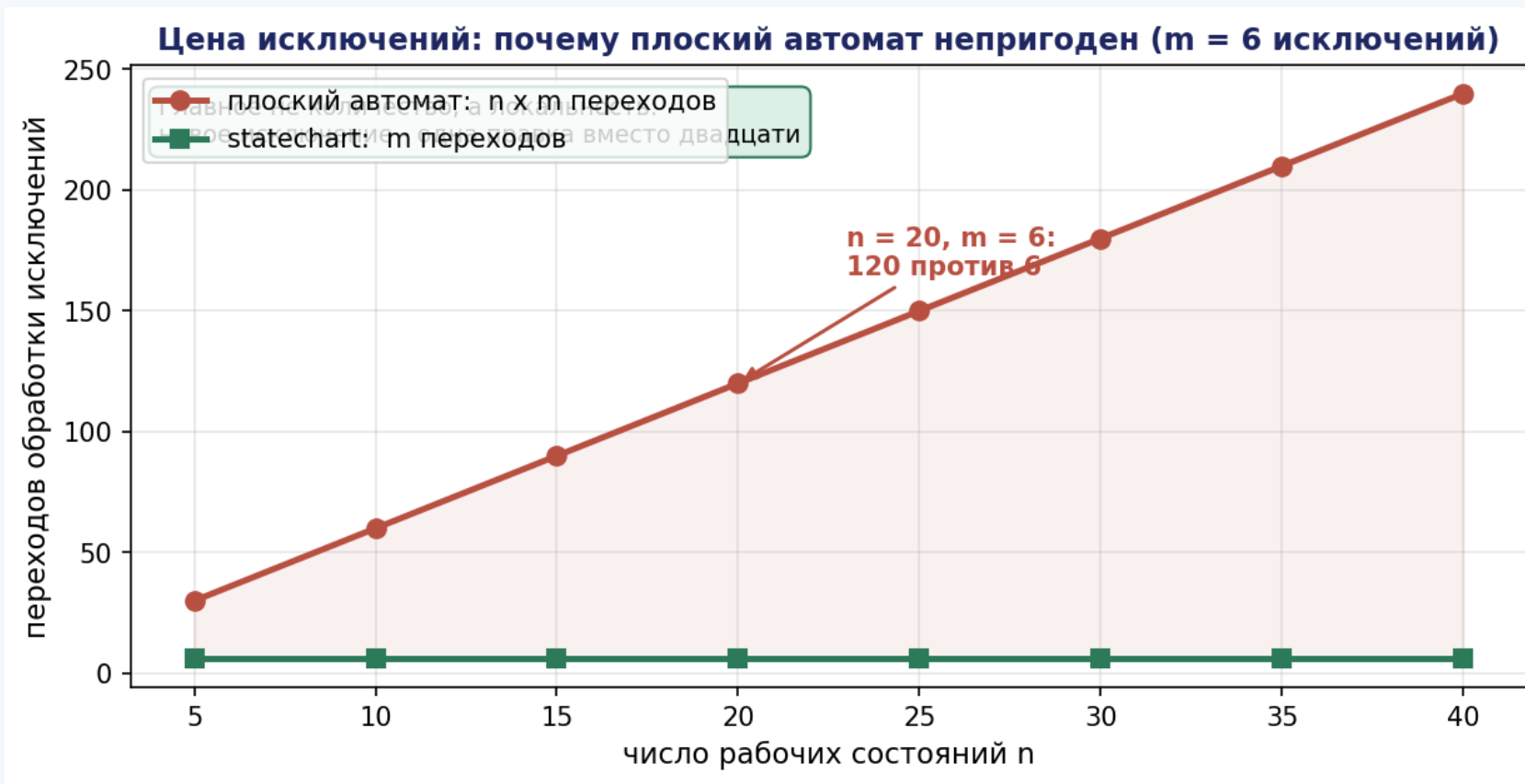
$$|\delta_exceptions| = n \cdot m$$

- n рабочих состояний, m исключительных ситуаций
- При $n = 20$, $m = 6$: 120 переходов сверх примерно 25 рабочих
- Добавление одного исключения требует n изменений
- Каждое из них может быть пропущено

Пропущенные переходы обработки исключений — самый частый класс дефектов исполнительной логики

6.1

Цена обработки исключений



120 переходов против 6. Но главное - локальность правки, а не количество.

6.2 Вторая беда: возврат

- После обработки исключения нужно вернуться туда, откуда вышли
- Плоский автомат не помнит, откуда пришёл
- Решение 1: дублировать состояния обработки — $n \cdot m$ состояний
- Решение 2: вспомогательные переменные — отказ от чисто автоматной модели
- Оба решения плохи по-своему

Факт

Именно на этом месте большинство проектов незаметно переходит от «автомата» к «автомату с флагами», после чего никакие формальные свойства уже не доказываются.

Statecharts: три расширения Харела

- **Иерархия (OR)**: переход из составного состояния наследуется вложенными
- **Параллелизм (AND)**: ортогональные области работают одновременно
- **Широковещание**: событие из одной области видимо во всех
- Плюс псевдосостояние **истории**: возврат в прерванное состояние
- Формула Харела: автоматы + иерархия + параллелизм + широковещание

$$n \cdot m \rightarrow m$$

- Переход из OR-состояния применим в любой вложенной конфигурации
- 120 переходов заменяются шестью
- Добавление исключения требует **одного** изменения вместо двадцати
- Изменение локально и не затрагивает рабочую логику

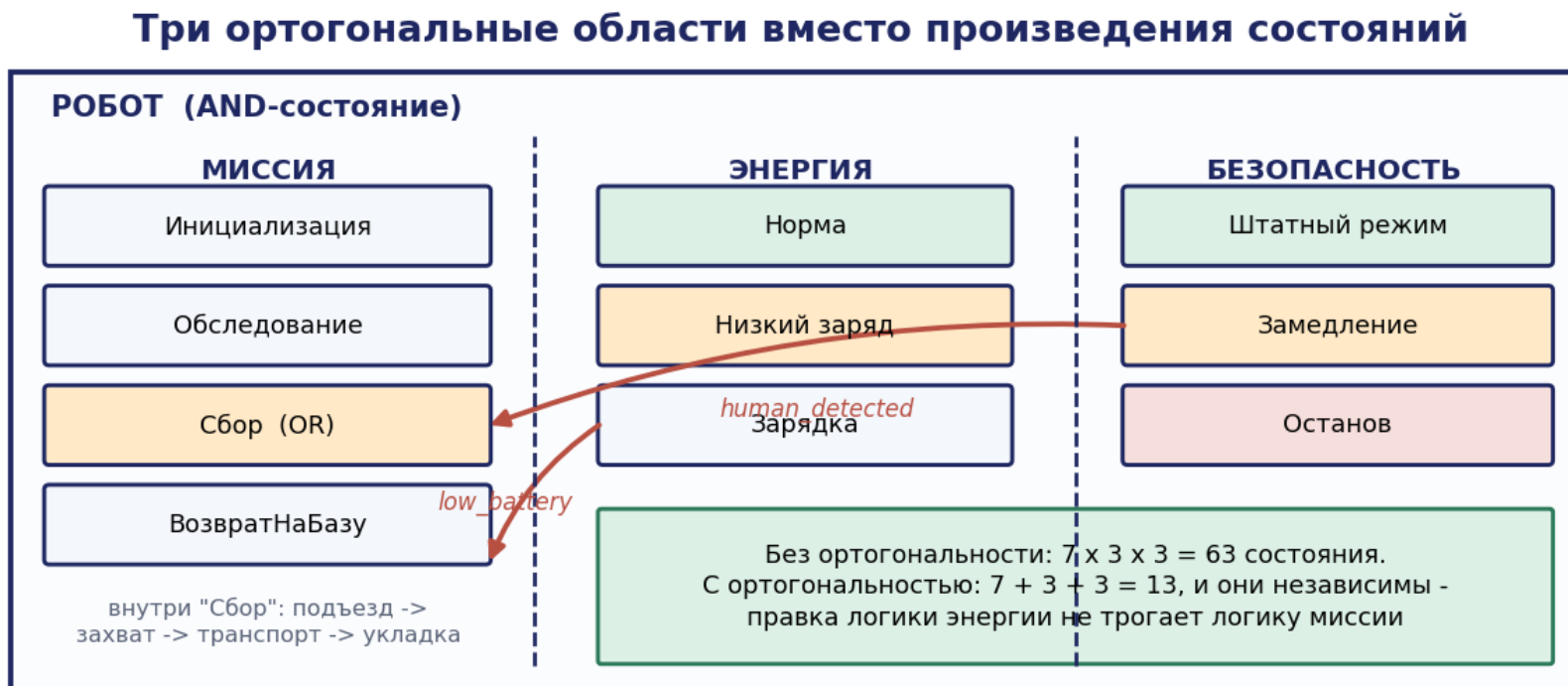
Качественный выигрыш важнее количественного: правка становится локальной

- **Миссия:** Инициализация → Обследование → Сбор → Возврат
- **Энергия:** Норма / Низкий заряд / Зарядка
- **Безопасность:** Штатный / Замедление / Останов
- Три области работают одновременно, координируются ширококешением
- Без ортогональности потребовалось бы $7 \times 3 \times 3 = 63$ состояния

Ключевая идея

Экономия структурная, а не только графическая: изменение логики энергопотребления не затрагивает логику миссии. Тринадцать состояний вместо шестидесяти трёх, и они независимы.

6.5 Три ортогональные области



13 состояний вместо 63, и они независимы: правка логики энергии не трогает логику миссии.

? Проверим

После зарядки робот должен вернуться к прерванной задаче. Какой механизм statecharts это обеспечивает?

А Вспомогательная переменная `last_state`

В Псевдосостояние истории (H или H*)

С Ортогональная область

Д Широковещательное событие

? Проверим

После зарядки робот должен вернуться к прерванной задаче. Какой механизм statecharts это обеспечивает?

A Вспомогательная переменная `last_state`

B Псевдосостояние истории (H или H*)

C Ортогональная область

D Широковещательное событие

Ответ. Псевдосостояние истории. Оно запоминает последнее активное вложенное состояние; H — на один уровень, H* — на всю глубину. Никаких вспомогательных переменных не требуется.

6.6 Ограничения statecharts

Ограничение	Суть	Последствие
Семантическая неоднозначность	более двадцати формальных семантик	модель ведёт себя по-разному в разных инструментах
Слабая композиционность	широковещание создаёт неявные связи	два независимо разработанных statechart не соединить
Переходы как глобальные связи	переход пересекает границы вложенности	структурный эквивалент оператора goto

Последние два ограничения и мотивировали переход робототехники к деревьям поведения

“

В автомате число возможных связей $O(n^2)$. В дереве рёбер
ровно $n - 1$.

Структурная причина того, почему деревья масштабируются лучше

- Дерево опрашивается сигналом-тиком от корня к листьям
- Каждый узел возвращает **Success**, **Failure** или **Running**
- Состояние не хранится в узлах — оно восстанавливается обходом
- Логика выражена структурой дерева, а не сетью переходов
- Sequence — «И» с прерыванием при неудаче; Fallback — «ИЛИ» с прерыванием при успехе

Ключевая идея

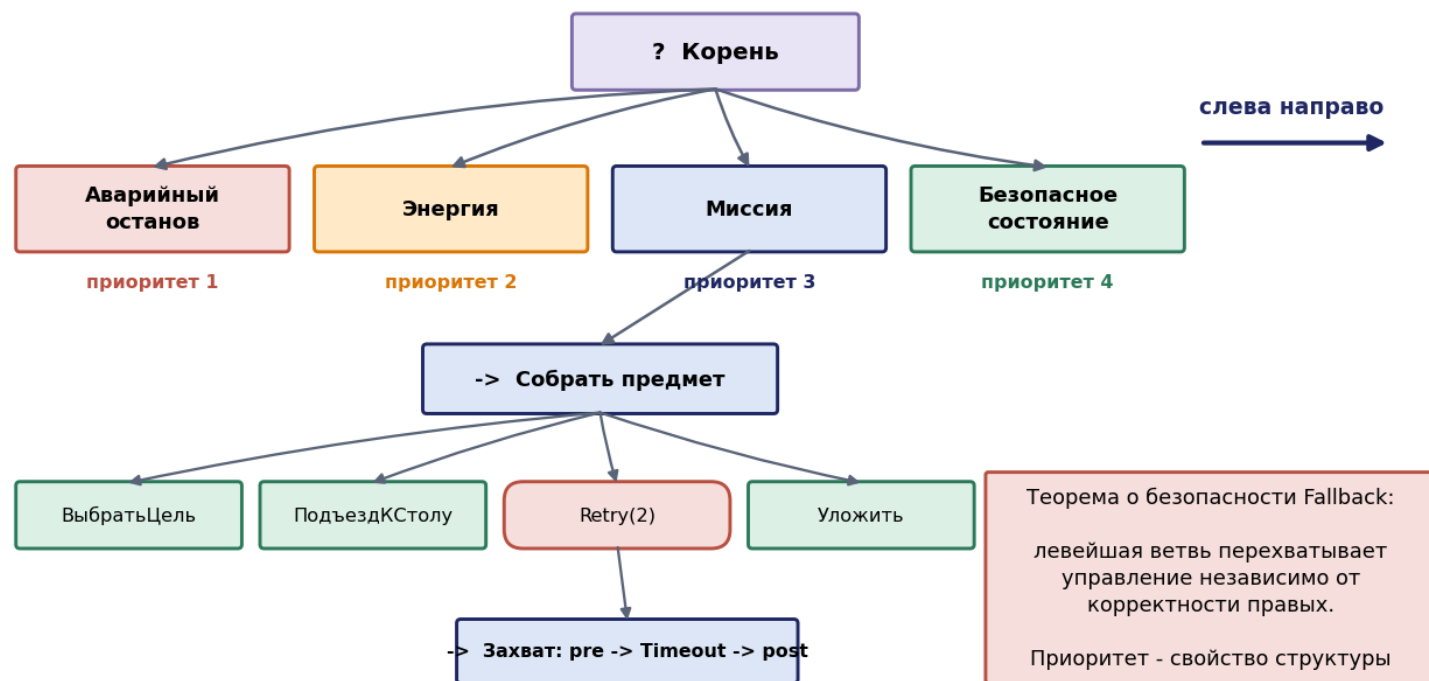
Статус Running — то, чего нет в обычных логических операциях. Именно он позволяет узлу сказать «я ещё работаю», и без него дерево не могло бы описывать длительные действия.

6.8 Типы узлов

Узел	Символ	Семантика
Sequence	→	Failure при первом Failure, Success если все Success
Fallback	?	Success при первом Success, Failure если все Failure
Parallel	⇒	Success при M успехах из N
Декоратор	ромб	модифицирует результат ребёнка: инверсия, повтор, таймаут
Action	прямоугольник	выполняет операцию, возвращает Running до завершения
Condition	овал	проверяет предикат, отвечает мгновенно

Шести типов узлов достаточно для любой исполнительной логики миссии

Дерево поведения миссии: приоритет обеспечивается положением узла



Приоритет защитного слоя обеспечен положением узла - это свойство структуры, а не дисциплины.

П Е Р Е Р Ы В

Конец первой части

Мы разобрали устройство деревьев. После перерыва — их формальные свойства и то, ради чего они нужны инженеру.

ЧАСТЬ 2

Свойства, которые можно доказать

Две теоремы композиции и их прямые следствия для проектирования.

$$T = (f_T, r_T, \Delta t), \quad x_{\{k+1\}} = f_T(x_k)$$

- f_T — функция перехода состояния за один тик
- r_T — функция результата: Success, Failure, Running
- Композиция определяется рекурсивно для Sequence и Fallback
- Области S_T, F_T, R_T образуют разбиение пространства состояний

Дерево — это динамическая система, и к нему применимы понятия устойчивости

6.10 Два понятия для теорем

- **FTS (успешность за конечное время)**: из области притяжения траектория достигает S_T за конечное время
- Не покидая при этом области притяжения
- **Безопасность относительно O** : траектория никогда не попадает в O
- Оба понятия относятся к дереву как к динамической системе
- Из них следуют композиционные свойства

6.11 Теорема о безопасности Fallback

Fallback(T_{safe} , T_{task}) безопасно независимо от T_{task}

- T_{task} получает тик только когда $r_{\{T_{safe}\}} = \text{Failure}$
- Если защита активируется до границы опасной области — переход в О невозможен
- Гарантия не зависит от корректности рабочей ветви
- Приоритет обеспечивается **положением узла**, а не проверкой

Структурная реализация принципа независимости защитного слоя

6.12 Теорема о последовательной композиции

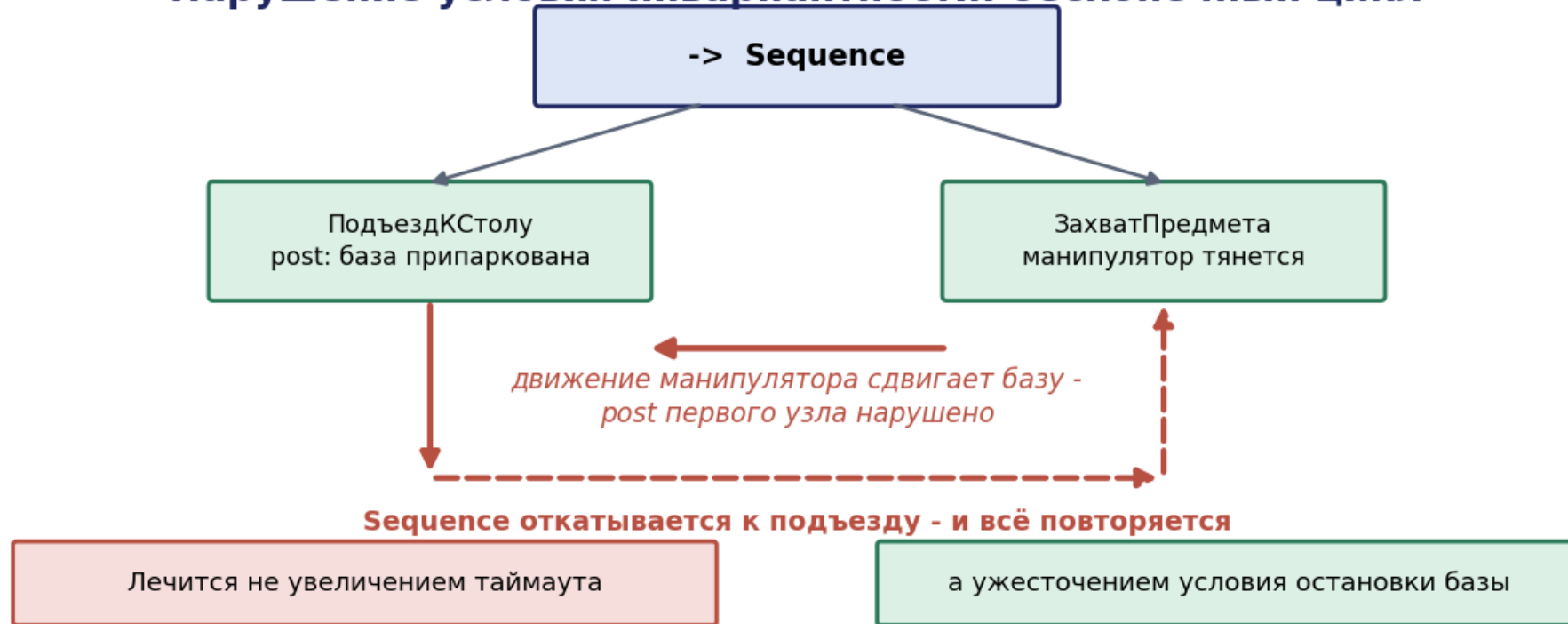
Sequence(T_1, T_2) FTS, если $S_{\{T_1\}}$ инвариантно относительно $f_{\{T_2\}}$

- T_1 достигает своей цели за конечное время τ_1
- T_2 достигает своей за τ_2 , не разрушая результат T_1
- Условие инвариантности — то, что нужно проверить
- Его нарушение даёт бесконечный цикл

Теорема указывает, что именно проверять при проектировании

6.12 Нарушение инвариантности

Нарушение условия инвариантности: бесконечный цикл



Классический бесконечный цикл подъезд - захват. Лечится не таймаутом, а условием остановки базы.

? Диагностируем

Робот бесконечно чередует подъезд к столу и попытку захвата. Что нарушено?

А

Неверный порядок узлов в Fallback

В

Инвариантность постусловия предыдущего действия при выполнении следующего

С

Слишком короткий таймаут захвата

Д

Отсутствие узла с памятью

? Диагностируем

Робот бесконечно чередует подъезд к столу и попытку захвата. Что нарушено?

- A Неверный порядок узлов в Fallback
- B Инвариантность постусловия предыдущего действия при выполнении следующего
- C Слишком короткий таймаут захвата
- D Отсутствие узла с памятью

Ответ. Второе: движение манипулятора нарушает условие «база припаркована», Sequence откатывается к подъезду. Исправление — ужесточить условие остановки базы или ослабить проверку с гистерезисом.

6.13 Теорема о выразительности

Всякий конечный автомат представим деревом поведения

- Конструкция: Fallback из ветвей $\text{Sequence}(\text{InState}(x_i), \text{BT_}x_i)$
- На каждом тике активна ровно одна ветвь
- Обратное вложение тоже есть, но автомат экспоненциален по глубине дерева
- Формализмы эквивалентны по выразительности, не по компактности

Также существуют вложения субсумпции, деревьев решений и телео-реактивных программ

6.14 Реактивность и её цена

- На каждом тике **все условия перепроверяются заново**
- Переключение происходит без описания перехода — оно следует из структуры
- Вычислительная цена: обход $O(|T|)$, при 200 узлах и 50 Гц несущественна
- Семантическая цена: каждое действие обязано быть корректно прерываемым
- Это ровно поле cancel контракта навыка из главы 1

Ключевая идея

Условия должны быть дешёвыми: узел-условие, запускающий планировщик или обработку изображения, выполняется на каждом тике и обрушивает частоту. Условия читают переменные, вычисления идут в действиях.

6.15 Память: когда она оправдана

- SequenceWithMemory не опрашивает детей, уже вернувших Success
- Blackboard — разделяемое хранилище результатов
- Оба вводят скрытое состояние и разрушают реактивность
- Правило: память только там, где перепроверка невозможна или дорога
- Каждое использование памяти документируется и обосновывается

Факт

Большинство дефектов деревьев локализуется именно в узлах с памятью: дерево перестаёт реагировать на изменение условий, а разработчик уверен, что оно реактивно.

Навык как поддерево: контракт становится структурой

- **pre** и **post** — узлы-условия в Sequence
- **inv** — охраняющая ветвь слева в Fallback
- **timeout** — декоратор Timeout
- **recovery** — декоратор RetryUntilSuccess или альтернативная ветвь
- **cancel** — обязательный метод прерывания в узле-действии

Ключевая идея

Все семь полей контракта из главы 1 получают прямое структурное выражение. Контракт — не абстракция, а описание того, что предстоит написать.

6.17 Обратное построение дерева из навыков



Самовосстановление без явного описания сценариев отказа

6.18 HFSM против BT: критерии выбора

Критерий	Иерархический автомат	Дерево поведения
Связи	явные переходы	только родитель — ребёнок
Модульность	ограничена	полная: поддереву самодостаточно
Реактивность	требует явных переходов	встроена
Приоритетное поведение	m переходов	один узел слева
Явные режимы работы	естественны	требуют явных условий
Верификация	развитые методы	переводы в автоматы, композиционные теоремы

Рекомендация курса: автомат режимов верхнего уровня + дерево поведения внутри автоматического режима

6.19 Шесть типовых ошибок

Ошибка	Как обнаружить
Защита правее рабочей ветви	проверить: для каждого критического условия ветвь левейшая?
Избыточная память на верхнем уровне	тест с внезапным изменением условия
Непрерываемые действия	тест с принудительным прерыванием на каждой фазе
Нарушение инвариантности (теорема 6.2)	анализ пар соседей в Sequence
Тяжёлые условия	профилирование частоты тика
Нет заведомо успешной правой ветви	поведение при Failure в корне не определено

Последняя ошибка самая коварная: дерево «работает», пока не исчерпаны все ветви

6.20 Реализация: инструменты

- **BehaviorTree.CPP** — XML, редактор Groot, ROS 2, промышленный стандарт
- **py_trees** — Python, визуализация, py_trees_ros
- **Nav2 BT Navigator** — навигация ROS 2 построена на деревьях
- **SMACH** — исторический HFSM для ROS 1, поддерживается, но не развивается
- Игровые движки Unreal и Unity — откуда пришла концепция

Факт

SMACH использовался в прежних версиях этого курса и остаётся педагогически полезным как реализация HFSM. Для новых проектов на ROS 2 отраслевой выбор — деревья поведения.

- **Перевод в автомат** — применимы методы главы 7, размер экспоненциален по глубине
- При реальной глубине 5–7 это приемлемо
- **Композиционные теоремы** — свойства выводятся локально, без глобальной модели
- **Мониторинг** — инварианты как узлы дерева, проверка на каждом тике
- Наиболее практичен второй путь

“

В дереве поведения приоритет защиты обеспечивается тем, что узел стоит левее. Это гарантия структуры, а не дисциплины программиста.

Теорема 6.1 в одной фразе

Выводы

- 1 Плоский автомат непригоден: $n \cdot m$ переходов обработки и отсутствие механизма возврата
- 2 Statecharts решают это иерархией, ортогональностью и историей — m переходов вместо $n \cdot m$
- 3 Дерево поведения хранит логику в структуре; состояние восстанавливается обходом
- 4 Реактивность следует из перепроверки условий, её цена — требование прерываемости действий
- 5 Приоритет защиты обеспечивается положением узла и не зависит от рабочих ветвей
- 6 Последовательная композиция требует инвариантности постусловия — иначе бесконечный цикл
- 7 Рекомендуемая схема: автомат режимов + дерево поведения внутри автоматического режима

Вопросы для самопроверки

1. Оцените число переходов обработки при $n = 30$, $m = 8$ и сравните со statechart
2. Зачем нужно псевдосостояние истории и какую проблему оно решает?
3. Чем AND-состояние отличается от произведения автоматов по объёму модели?
4. Почему у statecharts более двадцати семантик и каковы последствия?
5. Сформулируйте семантику Sequence и Fallback; каким логическим операциям они соответствуют?
6. Что означает Running и почему без него дерево не работало бы?
7. Докажите теорему о безопасности Fallback и объясните связь с принципом независимости защиты
8. Приведите пример нарушения условия инвариантности из теоремы о композиции
9. Почему memory=True разрушает реактивность и когда она оправдана?
10. Почему самая правая ветвь корневого Fallback должна быть заведомо успешной?